Clase 243 — Imágenes y contenedores seguros en el pipeline

> Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: *Securing DevOps* (Julien Vehent) y NIST SP 800-190 (Application Container Security Guide) ⏱ Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Construir imágenes de contenedor seguras dentro del pipeline: mínimas (distroless/multi-stage), sin ejecutar como root, escaneadas por vulnerabilidades y firmadas para garantizar su integridad. Una imagen es un artefacto que llega a producción; si está inflada, vulnerable o no verificada, es un vector directo de ataque. Usaremos **Trivy** para escanear, **hadolint** para el Dockerfile, y **cosign** para firmar.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Escribir** Dockerfiles seguros y mínimos con builds multi-stage y usuario no-root.
2. **Escanear** imágenes por vulnerabilidades de OS y de aplicación con Trivy.
3. **Lintar** Dockerfiles con hadolint aplicando buenas prácticas.
4. **Firmar y verificar** imágenes con cosign (keyless con OIDC).
5. **Aplicar** un gate que impida promover imágenes vulnerables o no firmadas.

🗺️ Temas

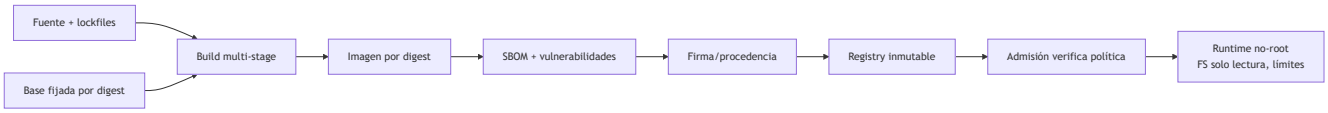
#	Tema	Por qué importa
1	Superficie de ataque de una imagen	Menos paquetes = menos CVE y menos exploits
2	Multi-stage builds y distroless	Imágenes mínimas de producción
3	Usuario no-root y capabilities	Limitar el daño de un contenedor comprometido
4	Escaneo con Trivy	Detectar CVE de OS y librerías
5	hadolint	Buenas prácticas del Dockerfile
6	Firma con cosign	Integridad y procedencia del artefacto
7	Gate de admisión	Solo imágenes firmadas y limpias a producción

🧠 Explicación en profundidad

La seguridad empieza por un artefacto reproducible y termina en runtime

Una imagen es un sistema de archivos por capas más metadatos de ejecución. Reducirla elimina paquetes y superficie, pero «mínima» no significa automáticamente segura: una aplicación vulnerable sigue siéndolo

en *distroless*, y una imagen sin shell puede ejecutarse como root o recibir secretos inseguros. El pipeline debe controlar el origen de la base, el contenido construido, la identidad del artefacto y la configuración con que se ejecutará.



El diagrama enseña una cadena de identidad. La etiqueta `app:1.0` es una referencia humana que puede apuntar a otro contenido; el digest identifica bytes concretos. La firma debe cubrir ese digest y la admisión debe verificar firma, emisor y política antes de ejecutar. Firmar no afirma que no existan vulnerabilidades: afirma quién autorizó un artefacto y permite vincularlo con procedencia.

Construcción: capas, secretos y base

Multi-stage separa herramientas de compilación del runtime, pero no borra automáticamente secretos copiados en una etapa o incluidos en el contexto. Se usan montajes de secretos del builder, `.dockerignore` y contextos mínimos; después se inspecciona el historial. La base se selecciona por necesidad, mantenimiento y compatibilidad, se fija por digest para reproducibilidad y se actualiza mediante un proceso controlado. Fijarla eternamente también conserva vulnerabilidades, por lo que bots o tareas de actualización proponen nuevos digests con pruebas.

El escaneo debe observar la imagen final y su SBOM. Un CVE en el stage de build importa de manera distinta a uno distribuido, aunque el constructor también puede ser atacado durante el build. Los resultados se triagean por alcance, exposición y política, como en la clase 240.

Ejecución: reducir consecuencias

El usuario no-root, sistema de archivos de solo lectura, capacidades Linux mínimas, seccomp, límites de recursos y ausencia de sockets privilegiados reducen el impacto. No constituyen una frontera equivalente a una VM en todos los escenarios: los contenedores comparten kernel. Las excepciones —escritura temporal, puerto o capacidad— se conceden en un punto específico, no desactivando todo el control.

Caso razonado: imagen pequeña con riesgo grande

Una imagen distroless de 30 MB pasa el escáner de paquetes, pero ejecuta como UID 0 y monta el socket de Docker para lanzar tareas. Un compromiso de la aplicación puede controlar el daemon del host. El equipo elimina el socket mediante una API de trabajos con permisos limitados, declara usuario no-root, fija la base, genera SBOM y verifica firma en admisión. El tamaño era una señal de mantenimiento; nunca fue una garantía de aislamiento.

📓 Glosario operativo

Término	Definición útil
Digest	Identificador criptográfico del contenido exacto de una imagen.
Multi-stage build	Construcción con etapas separadas para no distribuir todas las herramientas.
Distroless	Imagen de runtime sin distribución de propósito general; no implica invulnerabilidad.
Admisión	Decisión previa a ejecutar basada en identidad y política del artefacto.
Capacidad Linux	Privilegio granular que puede retirarse del proceso.

✅ Criterio de dominio

El alumno domina la clase cuando puede seguir una imagen desde su base hasta runtime, demostrar su identidad por digest, explicar qué cubren escaneo y firma, evitar secretos en capas y justificar controles de ejecución sin confundir tamaño con seguridad.

Cómo lo nombra el catálogo CWE y qué cambia al priorizar

Un archivo de construcción de contenedores es un buen sitio donde ver el **límite del catálogo CWE**. El `Dockerfile` vulnerable del laboratorio (`labs/devsecops-pipeline/repo-vulnerable`) contiene defectos con correspondencia **oficial** (un secreto en `ENV`, `CWE-798`), **inferida** (la imagen base `latest`, `CWE-1357`; `COPY . /app` que arrastra `.git` y credenciales, `CWE-538`; ejecutar como root, `CWE-250`; un puerto de depuración expuesto, `CWE-489`; instalar dependencias sin verificar integridad, `CWE-494`) y **ninguna** (`apt-get upgrade` dentro de la imagen, instalar paquetes sin limpiar la caché, herramientas innecesarias en el runtime, `CMD` en forma *shell*).

Que haya buenas prácticas sin CWE es una lección de honestidad: no todo mal hábito es una debilidad catalogada, y un informe que fuerce un identificador para parecer completo pierde credibilidad. El caso razonado de la clase, «imagen pequeña con riesgo grande», es justo el caso en que el tamaño no mide el riesgo, y las comprobaciones asociadas se tratan en la clase 366.

📖 Definiciones y características

- **Multi-stage build**: separar la fase de compilación de la imagen final. *Característica*: la imagen de producción no lleva compiladores ni herramientas de build.
- **Distroless**: imagen base sin shell ni gestor de paquetes, solo el runtime. *Característica*: reduce drásticamente la superficie de ataque.
- **Usuario no-root**: correr el proceso con un UID sin privilegios. *Característica*: un escape de aplicación no obtiene root en el contenedor.
- **Escaneo de imagen**: cruzar los paquetes de la imagen con bases de CVE. *Característica*: cubre capa de OS y dependencias de la app.
- **Firma (cosign)**: firma criptográfica de la imagen y su digest. *Característica*: verifica integridad y procedencia; keyless usa OIDC + transparency log.
- **Admission control**: política que solo admite imágenes que cumplen requisitos. *Característica*: bloquea en el registry o en Kubernetes lo no firmado/vulnerable.

🧰 Herramientas y preparación

- **Docker/BuildKit** o **buildah** para construir.
- **Trivy** para escanear imágenes.
- **hadolint** para lintar Dockerfiles.
- **cosign** (Sigstore) para firmar y verificar.

```
# Escanear una imagen:
trivy image --severity HIGH,CRITICAL miapp:1.0

# Lintar un Dockerfile:
docker run --rm -i hadolint/hadolint < Dockerfile

# Firmar (keyless con OIDC):
cosign sign miregistry/miapp@sha256:<digest>
```

🧪 Laboratorio guiado

> 🧪 **Laboratorio ejecutable del programa:** `devsecops-pipeline` — son las **capas 4 y 5** del lab: `hadolint` sobre el Dockerfile y `trivy` sobre el sistema operativo base.

1. **Parte de un Dockerfile "malo"** (imagen base pesada, root, sin pin) y lintéalo:

```
docker run --rm -i hadolint/hadolint < Dockerfile
```

2. **Reescríbelo seguro** con multi-stage, base distroless y usuario no-root:

```
# build stage
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app .

# runtime stage
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

3. **Escanea la imagen resultante** y compárala con la original:

```
trivy image --severity HIGH,CRITICAL miapp:seguro
```

Observa la caída de CVE al usar distroless. 4. **Firma la imagen con cosign** (keyless):

```
cosign sign --yes miregistry/miapp@sha256:<digest>
cosign verify miregistry/miapp@sha256:<digest> \
  --certificate-identity-regexp '.*' \
  --certificate-oidc-issuer-regexp '.*'
```

5. **Integra el gate en CI**. El job de build debe (a) lintar con hadolint, (b) escanear con Trivy y fallar con CRITICAL, (c) firmar solo si pasa, (d) publicar el digest.
6. **Admission en el clúster** (conceptual/opcional). Configura una política (cosign policy-controller o Kyverno) que solo admita imágenes firmadas por tu identidad.
7. **Genera SBOM de la imagen** (enlaza con clase 246):

```
trivy image --format cyclonedx -o sbom.json miapp:seguro
```

> Nota ética: escanear y endurecer imágenes propias es defensivo. No publiques imágenes con malware ni las uses para atacar; el laboratorio se hace con imágenes tuyas.

✍️ Ejercicios

1. Convierte un Dockerfile de una sola etapa a multi-stage y mide la reducción de tamaño.
2. Cambia una imagen base `ubuntu` por distroless y compara los CVE con Trivy.
3. Configura el contenedor para correr como usuario no-root y verifícalo.
4. Firma una imagen con cosign y verifica la firma.
5. Añade un gate de Trivy que rompa el build con vulnerabilidades CRITICAL.
6. Genera el SBOM de la imagen en formato CycloneDX.

📝 Reto verificable

Construye un pipeline que produzca una imagen mínima, escaneada y firmada.

Criterio de aceptación: (a) la imagen final usa multi-stage y base mínima/distroless y corre como no-root; (b) hadolint no reporta errores; (c) Trivy no reporta CVE CRITICAL y el gate falla si aparecen; (d) la imagen se firma con cosign y la firma se verifica; y (e) se genera y publica un SBOM de la imagen.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Imagen de 1.2 GB con cientos de CVE	Base pesada y build en una etapa. Usa multi-stage + distroless/alpine.
El contenedor corre como root	No definiste `USER`. Añade un usuario no-root o usa base `:nonroot`.
`cosign verify` falla	Identidad/issuer no coinciden. Ajusta `--certificate-identity` al del firmante real.
Trivy reporta CVE sin fix disponible	No hay parche aún. Documenta como riesgo aceptado/VEX y monitoriza.
La imagen firmada se puede sustituir por otra	Firmaste el tag, no el digest. Firma y despliega por digest inmutable.

❓ Preguntas frecuentes

❓ **¿Distroless o Alpine?** Distroless minimiza al máximo (sin shell, ideal para producción y para reducir superficie). Alpine es más pequeña que Debian y conserva shell/apk, útil para depurar. Elige según necesidad de troubleshooting vs mínima superficie.

❓ **¿Firmar la imagen impide que sea vulnerable?** No. La firma garantiza integridad y procedencia, no ausencia de CVE. Necesitas escanear además de firmar.

❓ **¿Debo escanear en build o en el registry?** Ambos. En build para bloquear temprano; en el registry de forma continua porque aparecen CVE nuevos sobre imágenes ya publicadas.

❓ **¿Keyless de cosign es seguro sin gestionar claves?** Sí: usa identidad OIDC efímera y un transparency log público (Rekor). Elimina la gestión de claves de larga vida, aunque puedes usar claves propias si lo prefieres.

🔗 Referencias

- NIST SP 800-190 Container Security — https://csrc.nist.gov/pubs/sp/800/190/final
- Trivy — https://trivy.dev/
- hadolint — https://github.com/hadolint/hadolint
- Sigstore cosign — https://docs.sigstore.dev/cosign/overview/
- Distroless images — https://github.com/GoogleContainerTools/distroless

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 242 — Seguridad en pipelines CI/CD

➡️ Siguiente clase

Clase 244 — Políticas como código con OPA